23AIE303 - Database Management Systems — Project Report

1. Project Title

Cloud-Native Distributed NewSQL Database for E-commerce, with Raft Consensus and MVCC Transactions

Batch A · Group 17

2. Team Members

#	Name	Roll number
1	Bhuvaneswaran S	CB.SC.U4AIE24012
2	Sriram S	CB.SC.U4AIE24066

GitHub repository:
https://github.com/sriram270506/Cloud-Native-Distributed-SQL-DB

3. Introduction and Problem Statement

3.1 The problem

An online store has to keep track of its customers, their addresses, the products it sells, the orders people place and the payments for those orders. Most college projects keep all of this in one database on one computer, which causes three problems:

- **One crash stops the shop.** If that single computer fails, nobody can buy anything, and orders that were being saved may be lost.
- **It cannot grow.** When more shoppers arrive, the only option is to buy a bigger, more expensive computer.
- **Simultaneous buyers can clash.** If two people buy at the same moment, one of them might see half-saved information.

3.2 Our solution: a NewSQL database

NewSQL means a database that looks and feels like a normal SQL database (tables, keys, `SELECT` queries, transactions) but runs on several computers at once. Well-known examples are Google Spanner and CockroachDB. We built a small one ourselves in the Go language and used it to run an e-commerce database. It relies on three main ideas.

Idea	In simple words	What it gives the store
Horizontal scaling	Grow by adding more computers (nodes) instead of buying one bigger computer.	Our cluster has 3 nodes, and each holds a full copy of the data.
Raft consensus	The nodes vote to choose a leader. The leader copies every change to the others, and a change counts only when most nodes (2 of 3) have saved it.	If a node crashes, no saved order is lost. If the leader crashes, a new leader is chosen automatically in under 2 seconds.
MVCC transactions	Multi-Version Concurrency Control: each change makes a new version of a row instead of overwriting it, so readers see a clean snapshot.	Shoppers never see half-saved orders. A group of statements either all succeed or all fail.

3.3 Objectives

- Design a clean (normalized) e-commerce database with proper keys and rules.
- Make it run on 3 computers with Raft, so it survives a crash.
- Support SQL: create, insert, update, delete, search, join, group and summarise.
- Prove that everything works with automated tests and a live crash test.

4. Conceptual and Logical Database Design

4.1 Conceptual design: what the store needs to remember

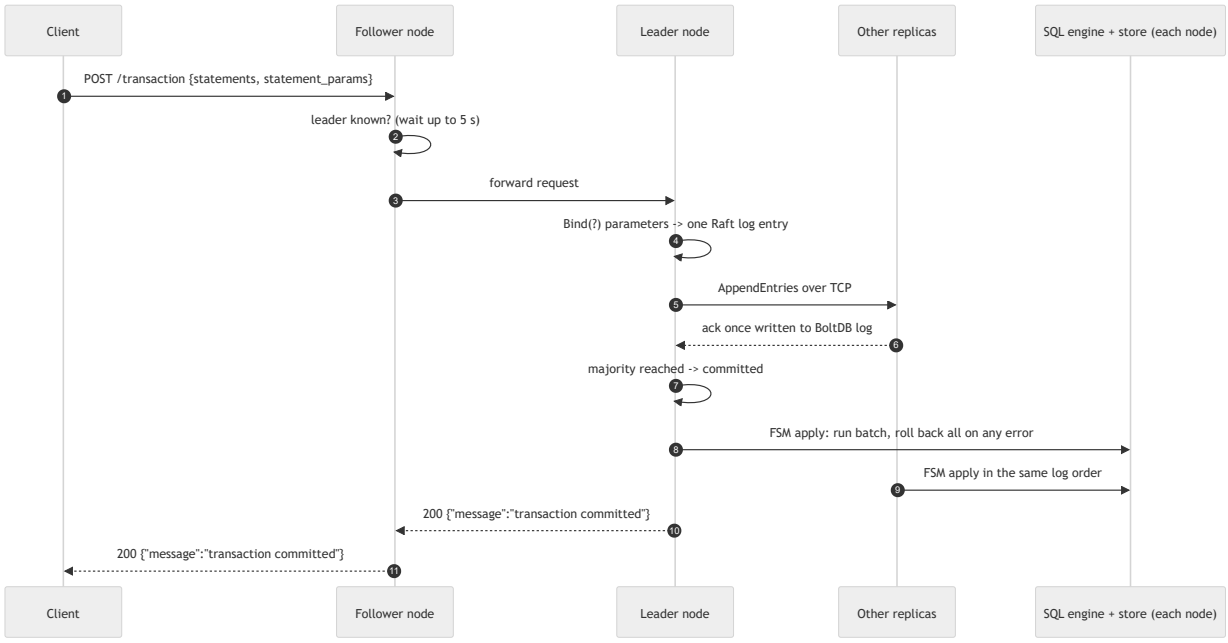
Entity (thing)	What it is	Unique ID
Customer	A person with an account	customer_id
Address	A home or work address of a customer	address_id
Product	Something we sell, with its price and stock	product_id
Order	One purchase by one customer on one date	order_id
Order Item	One product line inside an order (for example, 2 × Widget A)	order_item_id
Payment	Money paid for an order	payment_id

Relationship	Type	Meaning
Customer has Addresses	One-to-many	One customer can have many addresses.
Customer places Orders	One-to-many	One customer can place many orders; every order has exactly one customer.
Order contains Products	Many-to-many	An order has many products and a product appears in many orders. The order_items table sits in between.
Order receives Payments	One-to-many	An order can be paid in one or more payments.

4.2 Logical design: turning the entities into tables

- Every entity becomes a table, and its unique ID becomes the **primary key**.
- Every one-to-many relationship becomes a **foreign key** on the "many" side. For example, `orders.customer_id` points to `customers`.
- The many-to-many link between orders and products becomes its own table, `order_items`. It allows each product only once per order.
- **CHECK** rules stop bad data, such as negative prices, a quantity of 0 or an unknown order status.
- **ON DELETE CASCADE**: deleting a customer also deletes their addresses, orders, order items and payments. A product that has been sold cannot be deleted, so sales history stays correct.

4.3 How one order is saved across the computers



5. Normalization and Functional Dependencies

Normalization means splitting data into tables so that every fact is stored in exactly one place. To see why this matters, imagine the store kept everything in one big sheet:

```
order_id | date       | customer | email           | products ordered                | paid
1        | 2024-06-01 | Emma     | emma@example.com | Widget A x1 (19.99), Accessory 1 x2 | card
```

This sheet causes three kinds of trouble:

- **Update problem.** If Emma changes her email, every one of her orders must be edited. Missing one leaves the data disagreeing with itself.
- **Insert problem.** A new product cannot be added until somebody orders it.
- **Delete problem.** Deleting Emma's only order also deletes everything we knew about Emma.

5.1 How our tables meet each normal form

Normal form	The rule in plain words	How our e-commerce database follows it
1NF (First Normal Form)	Every cell holds one value. No lists inside a cell. Every row can be identified.	The 'products ordered' list became separate rows in order_items: an order with 3 products has 3 rows. Every table has a primary key.
2NF (Second Normal Form)	Already in 1NF, and every column depends on the whole key, not just part of it.	Product name, category and price depend only on product_id, so they live in products. order_items keeps just product_id. Every table has a one-column key, so a 'half-key' dependency cannot happen.
3NF (Third Normal Form)	Already in 2NF, and no column depends on another non-key column.	orders stores only customer_id, never the customer's email or city. Those depend on the customer, not the order, so they live in customers and addresses. Payment details live in payments.

> **Result:** every fact (an email, a price, an address) is stored exactly once. Changing it in one place updates it everywhere. Our tables actually meet BCNF, which is slightly stricter than 3NF: in every table, only a key decides the other columns.

5.2 Functional dependencies

A functional dependency **X → Y** means: if you know X, there is exactly one possible Y. For example, knowing `product_id = 1` tells you its name is "Widget A" and its price is 19.99. In our tables, X is always a key, which is what makes them normalized.

Table	If you know... (X)	...you know exactly (Y)
customers	customer_id, or email	first_name, last_name, email, created_at, updated_at, status
addresses	address_id, or (customer_id + type)	customer_id, line1, line2, city, state, postal_code, country, type
products	product_id, or product_name	product_name, category, unit_price, stock_qty, created_at, updated_at
orders	order_id	customer_id, order_date, total_amount, status, created_at, updated_at
order_items	order_item_id, or (order_id + product_id)	order_id, product_id, quantity, unit_price, line_total, created_at
payments	payment_id, or transaction_ref	order_id, method, amount, paid_at, status, transaction_ref

Some stored values are copies that could be worked out from other data. We keep them on purpose and guard them:

- `line_total` is checked by a CHECK rule to equal quantity × price.

- `order_items.unit_price` is the price paid on the day of the sale, which can differ from today's catalogue price.
- `total_amount` on each order is checked against the sum of its lines by query Q9.

6. Advanced SQL Implementation and Query Design

6.1 Building the database (DDL)

The tables are created by `database/schema.sql`, which contains:

- 6 tables, each with a **PRIMARY KEY**;
- **FOREIGN KEY** links with **ON DELETE CASCADE**;
- **UNIQUE** rules (email, product name, transaction reference);
- **CHECK** rules (prices > 0, valid status values, line_total = quantity × price);
- indexes for speed and one **view**.

`database/seed.sql` fills the tables with sample data: 10 customers, 10 addresses, 10 products, 10 orders, 12 order items and 10 payments.

6.2 The 14 documented queries

All queries are in `database/queries.sql`, each with a comment saying what it does. They cover multi-table joins, subqueries, GROUP BY with aggregates, HAVING, a window function, a view, CASE and UNION.

#	What it shows
Q1	Multi-table join (4 tables): every order line with customer, product and order date
Q2	GROUP BY with aggregates: revenue, units and distinct orders per category
Q3	GROUP BY ... HAVING: categories whose revenue exceeds 100
Q4	Correlated subquery: customers who spent more than the average order value
Q5	NOT EXISTS subquery: products that have never been ordered
Q6	IN subquery: customers who paid at least once with PayPal
Q7	LEFT JOIN + aggregate: order count and spend for every customer, including those with none
Q8	Common table expression + window function: rank products by revenue within each category
Q9	Data-integrity check: orders whose stored total differs from the sum of their lines
Q10	CASE expression: classify customers into spending tiers
Q11	Scalar subquery in SELECT: each product's price compared to its category average
Q12	View usage: orders that are fully paid, read through the order_summary view
Q13	UNION: one contact list of cities from home and work addresses, labelled by type
Q14	Date filtering + aggregates: daily revenue in the first week of June 2024

6.3 Four example queries with their real output

Q2. GROUP BY with aggregates: revenue, units and distinct orders per category

category-level sales report for the store manager.

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS orders,
       SUM(oi.quantity)            AS units_sold,
       ROUND(SUM(oi.line_total), 2) AS revenue,
       ROUND(AVG(oi.unit_price), 2) AS avg_unit_price
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

category	orders	units_sold	revenue	avg_unit_price
devices	3	3	699.97	233.32
tools	2	3	149.97	49.99
widgets	2	2	49.98	24.99
gadgets	2	2	24.98	12.49
accessories	3	4	21.96	5.66

Q4. Correlated subquery: customers who spent more than the average order value

the inner query is evaluated against the outer customer row.

```
SELECT c.customer_id,
       c.first_name || ' ' || c.last_name AS customer,
       (SELECT ROUND(SUM(o.total_amount), 2) FROM orders o WHERE o.customer_id = c.customer_id) AS total_spent
FROM customers c
WHERE (SELECT SUM(o.total_amount) FROM orders o WHERE o.customer_id = c.customer_id)
      > (SELECT AVG(total_amount) FROM orders)
ORDER BY total_spent DESC;
```

customer_id	customer	total_spent
9	Isabella Martinez	299.99
4	Noah Brown	204.98
8	James Davis	199.99
6	William Garcia	99.98

Q5. NOT EXISTS subquery: products that have never been ordered

anti-join to find dead stock.

```
SELECT p.product_id, p.product_name, p.category, p.stock_qty
FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id)
ORDER BY p.product_id;
```

product_id	product_name	category	stock_qty
6	Tool Beta	tools	40

Q8. Common table expression + window function: rank products by revenue within each category

RANK() OVER (PARTITION BY ...) computes a per-group ranking without collapsing rows.

```
WITH product_revenue AS (
  SELECT p.category, p.product_name, COALESCE(SUM(oi.line_total), 0) AS revenue
  FROM products p
  LEFT JOIN order_items oi ON oi.product_id = p.product_id
  GROUP BY p.category, p.product_name
)
SELECT category,
       product_name,
       ROUND(revenue, 2) AS revenue,
       RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rank_in_category
FROM product_revenue
ORDER BY category, rank_in_category;
```

category	product_name	revenue	rank_in_category
accessories	Accessory 1	14.97	1
accessories	Accessory 2	6.99	2
devices	Device One	399.98	1
devices	Device Two	299.99	2
gadgets	Gadget Y	14.99	1
gadgets	Gadget X	9.99	2

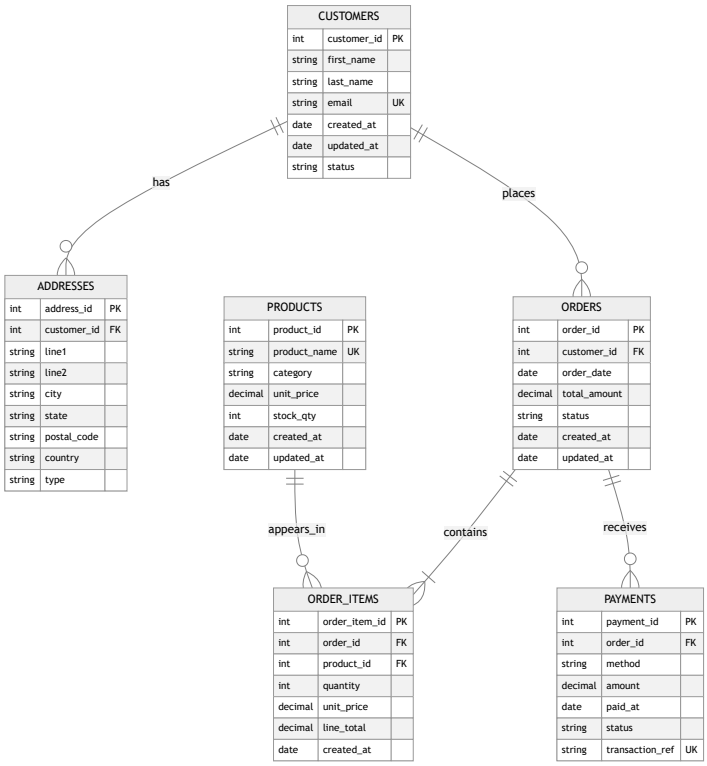
6.4 Transactions and safe queries on our distributed database

- **Transactions.** Several statements (for example: create the order, add its items, record the payment) are sent together. Raft copies them as one unit, and every node runs them. If any statement fails, all of them are undone on every node, thanks to MVCC.
- **Parameterized queries.** The user's input is sent separately from the SQL text, as `WHERE email = ?` plus the value. A hacking attempt like `x' OR '1'='1` is treated as a plain piece of text and matches 0 rows. Our tests check this.

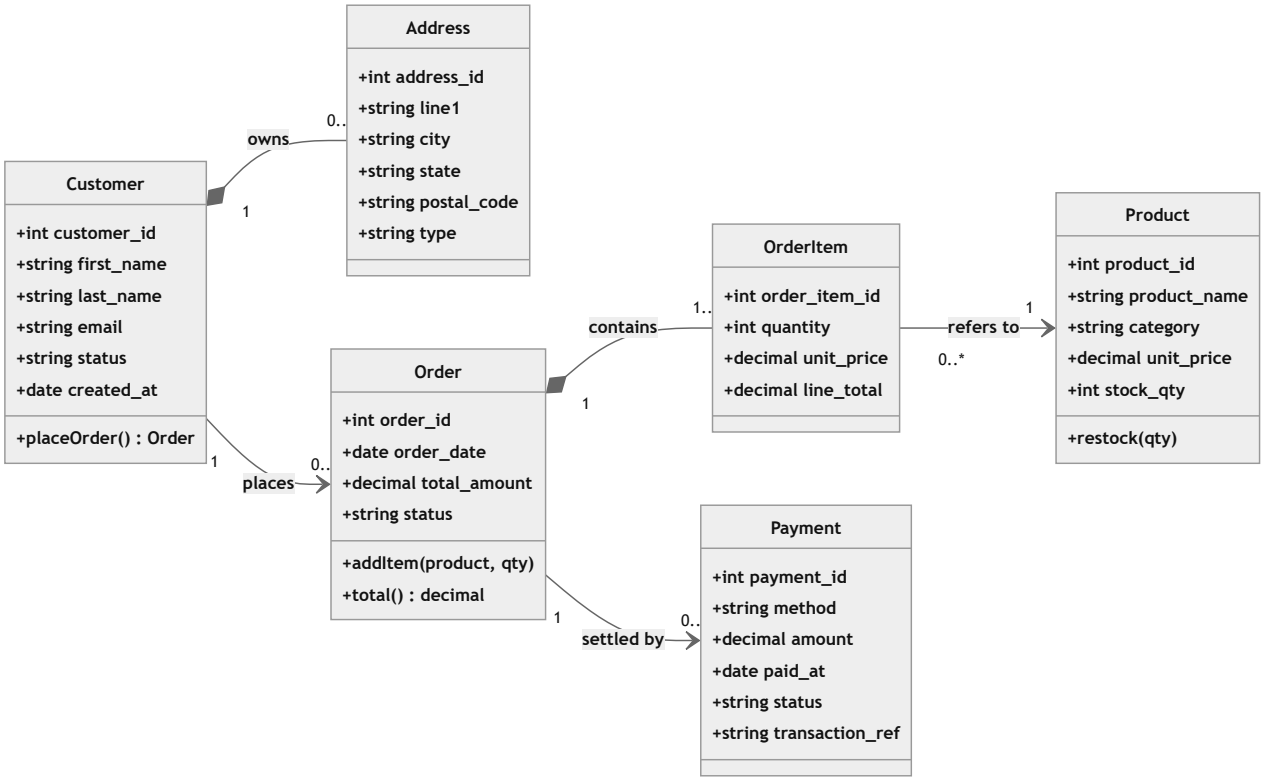
7. ER Diagram and UML Diagram

7.1 ER diagram

Boxes are tables. PK marks the primary key, FK a foreign key and UK a unique column. The line ends show how many: one, or many.



7.2 UML class diagram



8. Relational Schema

Notation: underlined = primary key, *italic* = foreign key (with the table it points to).

```
CUSTOMERS   (customer_id, first_name, last_name, email UNIQUE, created_at, updated_at, status)
ADDRESSES   (address_id, customer_id → CUSTOMERS, line1, line2, city, state, postal_code, country, type)
PRODUCTS    (product_id, product_name UNIQUE, category, unit_price, stock_qty, created_at, updated_at)
ORDERS      (order_id, customer_id → CUSTOMERS, order_date, total_amount, status, created_at, updated_at)
ORDER_ITEMS (order_item_id, order_id → ORDERS, product_id → PRODUCTS, quantity, unit_price, line_total,
created_at)
PAYMENTS    (payment_id, order_id → ORDERS, method, amount, paid_at, status, transaction_ref UNIQUE)
```

Foreign key	Points to	When the parent row is deleted
addresses.customer_id	customers	Addresses are deleted too (CASCADE)
orders.customer_id	customers	Orders are deleted too (CASCADE)
order_items.order_id	orders	Order lines are deleted too (CASCADE)
order_items.product_id	products	Blocked: a product that was sold cannot be deleted (RESTRICT)
payments.order_id	orders	Payments are deleted too (CASCADE)

9. Database Tables

The structure below is read back directly from the created database. All sample records are in `data/sample_db/`.

9.1 customers

Column	Type	Required	Key
customer_id	INTEGER	yes	PK
first_name	VARCHAR(50)	yes	
last_name	VARCHAR(50)	yes	
email	VARCHAR(120)	yes	UNIQUE
created_at	DATE	yes	
updated_at	DATE	yes	
status	VARCHAR(10)	yes	

Sample records (3 of 10)

customer_id	first_name	last_name	email	created_at	updated_at	status
1	Emma	Johnson	emma.johnson@example.com	2024-01-10	2024-06-01	active
2	Liam	Smith	liam.smith@example.com	2024-02-12	2024-06-02	active
3	Olivia	Williams	olivia.williams@example.com	2024-03-05	2024-06-03	active

9.2 addresses

Column	Type	Required	Key
address_id	INTEGER	yes	PK
customer_id	INTEGER	yes	FK → customers
line1	VARCHAR(100)	yes	
line2	VARCHAR(100)	no	
city	VARCHAR(60)	yes	
state	CHAR(2)	yes	
postal_code	VARCHAR(10)	yes	
country	VARCHAR(60)	yes	
type	VARCHAR(10)	yes	

Sample records (3 of 10)

address_id	customer_id	line1	line2	city	state	postal_code	country	type
1	1	123 Maple St		Springfield	IL	62701	USA	home
2	2	456 Oak Ave		Madison	WI	53703	USA	home
3	3	789 Pine Rd		Columbus	OH	43085	USA	home

9.3 products

Column	Type	Required	Key
product_id	INTEGER	yes	PK
product_name	VARCHAR(100)	yes	UNIQUE
category	VARCHAR(40)	yes	
unit_price	NUMERIC(10, 2)	yes	
stock_qty	INTEGER	yes	
created_at	DATE	yes	
updated_at	DATE	yes	

Sample records (3 of 10)

product_id	product_name	category	unit_price	stock_qty	created_at	updated_at
1	Widget A	widgets	19.99	100	2024-01-01	2024-06-01
2	Widget B	widgets	29.99	80	2024-01-05	2024-06-02
3	Gadget X	gadgets	9.99	200	2024-02-01	2024-06-03

9.4 orders

Column	Type	Required	Key
order_id	INTEGER	yes	PK
customer_id	INTEGER	yes	FK → customers
order_date	DATE	yes	
total_amount	NUMERIC(10, 2)	yes	
status	VARCHAR(10)	yes	
created_at	DATE	yes	
updated_at	DATE	yes	

Sample records (3 of 10)

order_id	customer_id	order_date	total_amount	status	created_at	updated_at
1	1	2024-06-01	29.97	completed	2024-06-01	2024-06-02
2	2	2024-06-02	29.99	completed	2024-06-02	2024-06-03
3	3	2024-06-03	9.99	completed	2024-06-03	2024-06-04

9.5 order_items

Column	Type	Required	Key
order_item_id	INTEGER	yes	PK
order_id	INTEGER	yes	FK → orders
product_id	INTEGER	yes	FK → products
quantity	INTEGER	yes	
unit_price	NUMERIC(10, 2)	yes	
line_total	NUMERIC(10, 2)	yes	
created_at	DATE	yes	

Sample records (3 of 12)

order_item_id	order_id	product_id	quantity	unit_price	line_total	created_at
1	1	1	1	19.99	19.99	2024-06-01
2	1	7	2	4.99	9.98	2024-06-01
3	2	2	1	29.99	29.99	2024-06-02

9.6 payments

Column	Type	Required	Key
payment_id	INTEGER	yes	PK
order_id	INTEGER	yes	FK → orders
method	VARCHAR(15)	yes	
amount	NUMERIC(10, 2)	yes	
paid_at	DATE	yes	
status	VARCHAR(10)	yes	
transaction_ref	VARCHAR(30)	yes	UNIQUE

Sample records (3 of 10)

payment_id	order_id	method	amount	paid_at	status	transaction_ref
1	1	card	29.97	2024-06-01	paid	tx1001
2	2	card	29.99	2024-06-02	paid	tx1002
3	3	paypal	9.99	2024-06-03	paid	tx1003

10. Conclusion and Future Scope

10.1 Conclusion

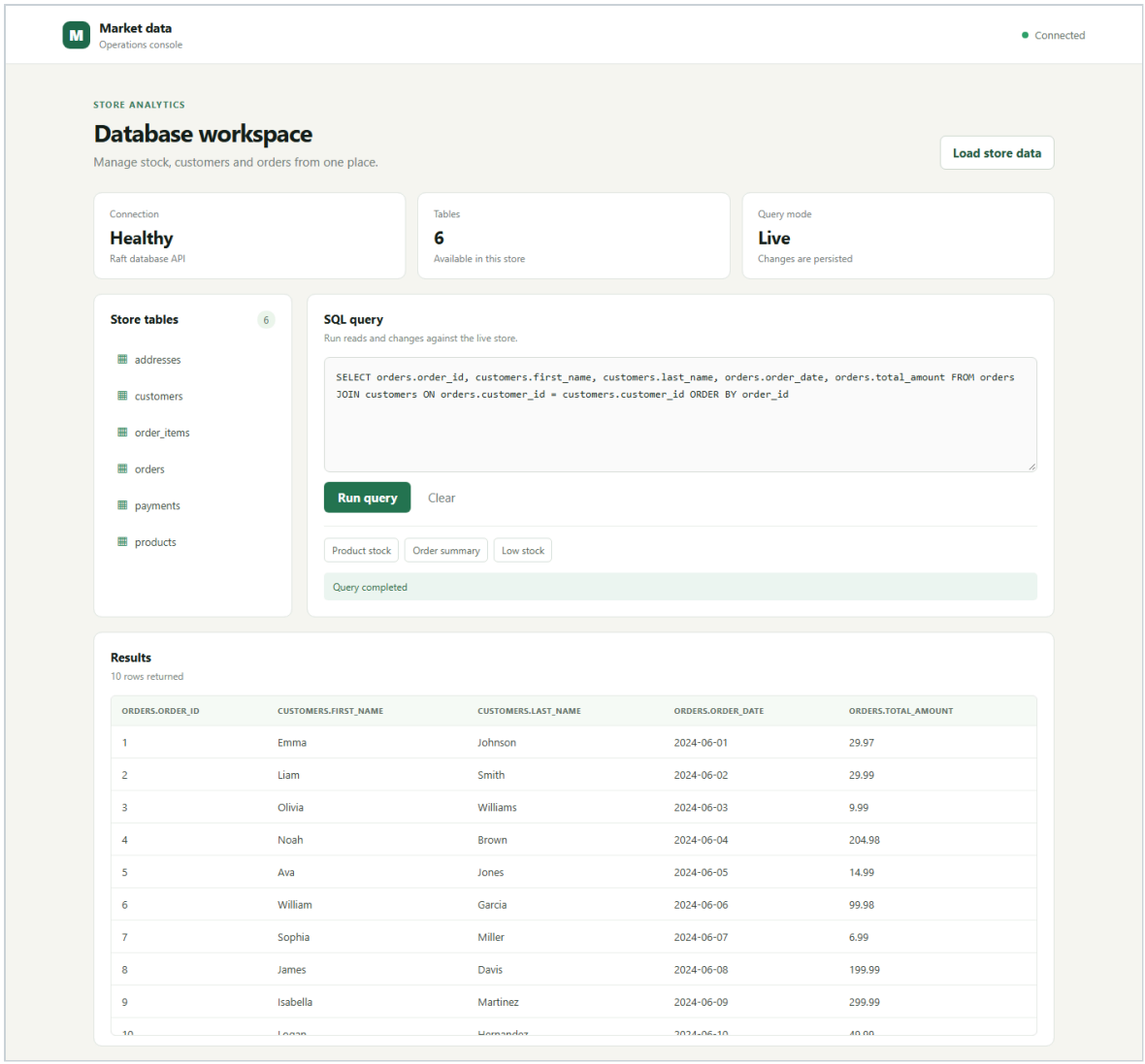
- We built a working **NewSQL database**: normal SQL tables, copied across 3 computers.
- **Raft** makes sure a change is saved only when most computers agree. In our crash test we killed the leader, a new leader took over in under 2 seconds, and no order was lost.
- **MVCC transactions** let shoppers read clean, consistent data while others buy, and make a group of changes all-or-nothing.
- The e-commerce schema is **normalized (3NF, in fact BCNF)** and protected with primary keys, foreign keys, UNIQUE and CHECK rules.
- Everything is checked automatically by Go tests, 16 SQL tests and a live failover demo (see the appendix).

10.2 Future scope

- Enforce foreign keys, UNIQUE and CHECK rules inside our own distributed engine, and support subqueries there too.
- Split large tables across different groups of computers (sharding), so writes also scale with more machines.
- Add logins, passwords and encrypted connections.

Appendix: Testing and Demonstration

Test	Command	What it proves
Engine tests (Go)	go test ./...	MVCC isolation; SQL commands; SQL-injection safety; 3 computers copy data, survive a leader crash and recover
Database tests (16)	python -m unittest discover -s database -v	Keys, foreign keys, UNIQUE and CHECK rules reject bad data; CASCADE deletes work; every query gives the right answer
Live demo	scripts/cluster.ps1, then python database/seed_cluster.py	Starts 3 computers, loads the e-commerce data and runs queries on them



Web console running a JOIN on the live 3-node cluster